Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on Laboratory Work No. 5

The File System with Your Own Hands

Operating Systems

Done by:
Student, group FAF-241
Cațț Chiril

Checked by:
asist. univ. Reițman Patricia

Chișinău – 2026

Contents

Purpose and environment **1**

1 The directory tree and file types **1**

2 Inodes, links, and allocated space **1**

3 Permissions **2**

4 Disks and the kernel as files **3**

Conclusion **4**

Purpose and environment

The goal was to inspect Linux path types, inode and link behavior, access permissions, disk layout, and kernel information exposed through `/proc`. I ran the experiments on 12 October 2026 as the normal user `class` in the installed Ubuntu guest under UTM, not as root. The commands ran in a new directory beneath `/home/class/os-lab5`. Their full output is preserved in `lab5-results.txt`; the relevant commands and outputs appear below. The CPU, RAM and disk figures describe this virtual machine.

1 The directory tree and file types

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx ... /bin -> usr/bin
crw-rw-rw- ... /dev/null
drwxr-xr-x ... /etc
-rw-r--r-- ... /etc/passwd
drwxrwxrwt ... /tmp
$ ls -l /dev/vda
brw-rw---- ... /dev/vda
$ du -sh /var/log /usr
33M   /var/log
5.7G  /usr
```

The leading `l` marks `/bin` as a symbolic link to `usr/bin`, so it resolves to `/usr/bin`. The leading `c` marks `/dev/null` as a character device, while `/dev/vda` starts with `b` and is a block device. The leading `d` marks `/etc` and `/tmp` as directories; the leading `-` marks `/etc/passwd` as a regular file. The `file` command also reported `/etc/passwd` as ASCII text and `/bin` as a symbolic link. `/usr` occupies about 5.7 GiB versus 33 MiB for `/var/log`: roughly 5.7 GiB more, or around 177 times as much. This reflects installed programs and libraries in `/usr` compared with current logs in `/var/log`.

2 Inodes, links, and allocated space

```
$ echo hello > notes.txt; stat notes.txt
Size: 6   Blocks: 8   IO Block: 4096   regular file
Inode: 1056330   Links: 1
$ ln notes.txt hard.txt; ls -li notes.txt hard.txt
1056330 ... 2 ... hard.txt
1056330 ... 2 ... notes.txt
$ rm notes.txt; cat hard.txt; ls -li hard.txt
hello
1056330 ... 1 ... hard.txt
```

```
$ ln -s hard.txt soft.txt; ls -li hard.txt soft.txt
1056330 ... hard.txt
1061575 ... soft.txt -> hard.txt
$ rm hard.txt; cat soft.txt
cat: soft.txt: No such file or directory
$ stat soft.txt
Size: 8   Blocks: 0   symbolic link
```

`Inode: 1056330` identifies the file's inode. `Links: 1` is its hard-link count: the number of directory entries naming that inode. Creating `hard.txt` raised the count to two and gave both names the same inode. `rm notes.txt` removed one directory entry, not the file data; `hard.txt` still named the inode and read `hello`. The symbolic link had its own inode, 1061575. Its contents were the eight bytes of the relative path `hard.txt`. After that target name was removed, the symlink remained but its path resolved nowhere, so `cat soft.txt` failed.

```
$ mkdir d; ls -ld d
drwxrwxr-x 2 class class ... d
$ mkdir d/x d/y; ls -ld d
drwxrwxr-x 4 class class ... d
$ echo x > one.txt; ls -l one.txt; du -h one.txt
-rw-rw-r-- ... 2 ... one.txt
4.0K  one.txt
$ truncate -s 1G big.img; ls -lh big.img; du -h big.img
-rw-rw-r-- ... 1.0G ... big.img
0  big.img
```

The new directory's count of two comes from its entry in the parent and its own `.` entry. Each new child directory has a `..` entry that points back to `d`, raising `d`'s count to four. The two-byte file consumes one 4 KiB filesystem block. `truncate` made a sparse 1 GiB file: it set the logical length without writing blocks into the hole, so `du` reported zero allocated space. The test image was removed after measurement.

3 Permissions

Mode	Permission string	Typical use
600	`rw----`	Private file readable and writable by its owner
644	`rw-r-r-`	Publicly readable document, owner writable
755	`rwxr-xr-x`	Executable program or traversable directory
700	`rwx---`	Private directory accessible only to its owner

```
$ chmod 600 s.txt; ls -l s.txt
-rw------- ... s.txt
```

```
$ chmod 000 s.txt; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt; cat s.txt
secret
$ umask; ls -ld new.txt newdir
0002
-rw-rw-r-- ... new.txt
drwxrwxr-x ... newdir
```

Ownership determines which permission class applies; it does not itself grant access. With mode 000, the owner's read bit was absent, so the normal user could not read `s.txt`. Restoring `u+r` made it readable. The umask `0002` removes the other-write bit from the default creation modes: `0666 & ~0002 = 0664` for the new file and `0777 & ~0002 = 0775` for the new directory.

```
$ chmod u=r,go= box; ls box; cat box/f
f
cat: box/f: Permission denied
$ chmod u=x,go= box; ls box; cat box/f
ls: cannot open directory 'box': Permission denied
hi
$ ls -l /usr/bin/passwd; ls -ld /tmp
-rwsr-xr-x ... root root ... /usr/bin/passwd
drwxrwxrwt ... root root ... /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- ... root shadow ... /etc/shadow
```

For a directory, read permission allows listing entry names, whereas execute permission allows searching or traversing it by a known name. Thus read-only `box` listed `f` but blocked `box/f`; execute-only `box` could not be listed, but `cat box/f` succeeded because the file itself was readable. I restored `box` to mode 755 afterward.

The `s` in `passwd`'s owner-execute position is the set-user-ID bit: the program executes with root's effective user ID. It needs that privilege to update password data while ordinary users cannot read `/etc/shadow`. The `t` in `/tmp` is the sticky bit. Because `/tmp` is shared and world-writable, it restricts deletion and renaming of entries to their owner, the directory owner, or root.

4 Disks and the kernel as files

```
$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
vda       64G disk
|-vda1     1G part vfat     /boot/efi
```

```
‘-vda2  62.9G part ext4     /
$ df -h /
Filesystem  Size  Used  Avail  Use%  Mounted on
/dev/vda2    62G   12G    48G   20%  /
$ findmnt -no SOURCE,FSTYPE,TARGET /
/dev/vda2 ext4 /
$ grep -c ’^processor’ /proc/cpuinfo
6
$ grep ’^MemTotal:’ /proc/meminfo
MemTotal:        3461284 kB
```

The virtual disk is `/dev/vda` (64 GiB). Its first partition, `/dev/vda1`, is a 1 GiB VFAT EFI partition mounted at `/boot/efi`. The 62.9 GiB second partition, `/dev/vda2`, contains the ext4 root filesystem. The root filesystem is 20% full, with 12 GiB used and 48 GiB available according to `df`. The VM exposes six logical processors and 3,461,284 kB of RAM, approximately 3.30 GiB.

```
$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   4301
Ngid:   0
$ ls -l /proc/self/fd
0 -> /dev/pts/0
1 -> pipe:[30151]
2 -> pipe:[30151]
3 -> /proc/4302/fd
```

`/proc/self` refers to the process that opens it. Here `head`, rather than the shell, opened `/proc/self/status`, so `Name` was `head`. Similarly, `ls -l /proc/self/fd` listed the file descriptors of that `ls` process: standard input was the terminal, standard output and error were the transcript pipe, and descriptor 3 was its open fd directory.

Conclusion

The experiments showed that path names and file data are separate: hard links share an inode, while symbolic links store a path that can become dangling. Logical file length can differ greatly from allocated disk space. Permission checks depend on mode bits and directory traversal rights, with setuid and sticky bits serving specific shared-system needs. Finally, disk mounts and `/proc` exposed the guest’s storage and runtime state through ordinary filesystem interfaces.